

Teoria_Esercizi_Esame

Teoria per gli Esercizi

1. Conversioni di Tipo, Promozioni e Valutazione delle Espressioni

1.1 Grado di Conversione dei Tipi (*Conversion Rank* C99 §6.3.1.1)

Ogni tipo intero possiede un rango gerarchico che stabilisce la precedenza nelle conversioni aritmetiche:

$$_Bool < char < short < int < long < long long$$

- **Tipi con e senza segno:** Un tipo `signed` e il rispettivo `unsigned` hanno lo **stesso rango** (`rank(int) == rank(unsigned int)`).
- **Varianti char:** `char`, `signed char` e `unsigned char` condividono lo stesso rango.
- **Enumerazioni:** I tipi `enum` hanno il rango di `int`.
- **Tipi Floating Point:** Sono sempre strettamente superiori a qualunque tipo intero:

$$float < double < long double$$

1.2 Promozioni Intere (*Integer Promotions* §6.3.1.1)

Prima di eseguire qualsiasi operazione aritmetica, logica o bitwise, i tipi con rango inferiore a `int` (`_Bool`, `char`, `signed char`, `unsigned char`, `short`, `unsigned short`) subiscono **sempre** la promozione automatica:

- a `int`: se `int` può rappresentare tutti i valori del tipo di partenza.
- a `unsigned int`: se `int` non basta (es. `unsigned short` su macchine a 16 bit).

Esempio: In `char a = 70, b = 70; a = (a * b) / 50;`, le variabili `a` e `b` vengono promosse a `int` prima del prodotto ($70 \times 70 = 4900$). Il risultato intermedio è `int` e viene riconvertito a `char` solo all'assegnamento finale.

1.3 Conversioni Aritmetiche Consuete

Quando un operatore binario (+, -, *, /, %, <, >, =, &, |, ^) agisce su operandi di tipo diverso:

1. **Presenza di Floating Point:** L'altro operando viene promosso al tipo floating point di grado più alto (long double > double > float).
2. **Operandi Interi (dopo le promozioni intere a int):**
 - **Stesso Segno:** L'operando con rango inferiore si converte a quello con rango superiore.
 - **Segno Diverso – Unsigned ≥ Signed:** Se il tipo unsigned ha rango maggiore o uguale al signed, il signed viene convertito a unsigned di quel rango.
 - *Esempio:* int e unsigned int → int convertito a unsigned int.
 - *Esempio:* int e unsigned long → int convertito a unsigned long.
 - **Segno Diverso – Signed > Unsigned:**
 - Se il tipo signed può rappresentare **tutti** i valori del tipo unsigned (es. long 64 bit e unsigned int 32 bit) → unsigned int convertito a long.
 - Se il tipo signed **non** può rappresentare tutti i valori dell'unsigned (es. entrambi a 32 o 64 bit) → entrambi convertiti a unsigned corrispondente al signed (es. unsigned long).

1.4 Assegnamenti, Parametri, Return e Operatore Ternario

- **Assegnamento (=):** Il valore destro viene convertito al tipo della variabile sinistra.
 - Floating point → intero: troncamento della parte decimale (-2.5L → -2).
 - Tipo più grande → tipo intero più piccolo: troncamento dei bit più significativi.
- **Chiamate e Return:** Gli argomenti vengono convertiti ai tipi dei parametri formali nel prototipo; il valore di return viene convertito al tipo di ritorno dichiarato della funzione.
- **Operatore Ternario (? :):** Il secondo e il terzo ramo subiscono le *Usual Arithmetic Conversions* per determinare il tipo comune dell'intera espressione.

1.5 Wrapping Unsigned e Precisione Floating Point

- **Aritmetica Modulare Unsigned (2^N):** L'overflow dei tipi `unsigned` è garantito dallo standard come operazione modulo ($UMAX + 1$):

$$\text{Valore Unsigned} = \text{valore} \pmod{2^N}$$

- Assegnando `-k` a un `unsigned short` (16 bit, `USHRT_MAX = 65535`):

$$-k \pmod{65536} = 65536 - k$$

- `-5L` assegnato a `unsigned short` $\rightarrow 65536 - 5 = 65531$.
- `-2L` assegnato a `unsigned short` $\rightarrow 65536 - 2 = 65534$.
- **Rappresentabilità esatta:**
 - `float` (32 bit, IEEE 754): 24 bit di mantissa ≈ 7 cifre decimali di precisione.
 - `double` (64 bit): 53 bit di mantissa ≈ 15 --17 cifre decimali di precisione.
 - Un intero fino a 2^{53} memorizzato in un `double` è **rappresentato in modo esatto al 100%**.

1.6 Esercizi d'Esame Risolti

Esercizio A (Confronto signed/unsigned):

```
int i = -1;
unsigned int limit = 200U;
if (i < limit) { ... }
```

- `i` (`int`, `-1`) viene convertito a `unsigned int` $\rightarrow 4294967295U$.
- $4294967295U < 200U$ è **falso** $\rightarrow$ il blocco `if` non viene eseguito.

Esercizio B (Conversioni a cascata e chiamate):

```
long int g2(unsigned long p) { return p + 'e' - 'a'; }
int g1(int p) {
    char c = 'k';           // 'k' = 107
    return g2(p + c - 'd'); // 'd' = 100
}
int main(void) {
    unsigned short x = -5L; // x = 65531
    double b = g1(x);       // b = 65542.0
}
```

1. `-5L` $\rightarrow$ `unsigned short x = 65531`.
2. In `g1(x)`: `x` promosso a `int` (`65531`). `c` promosso a `int` (`107`). $65531 + 107 - 100 = 65538$. Convertito a `unsigned long` per `g2`.
3. In `g2`: $'e'(101) - 'a'(97) = 4 \rightarrow 65538 + 4 = 65542UL$. Ritorna `long int 65542`.

4. `g1` ritorna `int` 65542 → assegnato a `double b = 65542.000000`.

2. Dichiarazioni, Definizioni, Linkage e Durata di Memorizzazione

2.1 Dichiarazione vs Definizione

- **Dichiarazione:** Fa conoscere identificatore e tipo senza allocare memoria o fornire il corpo (`extern int b;`, `int func(int x);`).
- **Definizione:** Alloca memoria o implementa la funzione (`int c = 1;`, `int func(int x) { return x; }`).
- **Definizione Tentativa (Tentative Definition §6.9.2):** Dichiarazione globale senza `extern/static` e senza inizializzatore (es. `int a;`). Se nel file non segue alcuna definizione con inizializzatore, funge da definizione effettiva con valore iniziale 0.

2.2 I Tre Tipi di Linkage (§6.2.2)

Linkage	Visibilità	Casi Tipici
Esterno (External)	Condiviso tra tutti i file del progetto (risolto dal linker).	Variabili globali non-static, funzioni non-static, variabili <code>extern</code> .
Interno (Internal)	Visibile solo nel file <code>.c</code> corrente .	Variabili globali <code>static</code> , funzioni <code>static</code> .
Nessuno (No Linkage)	Riferimento univoco e locale al blocco/funzione .	Parametri formali, variabili locali automatiche, variabili locali <code>static</code> .

Nota: Le variabili locali dichiarate `static` hanno **durata statica** (non perdono il valore tra chiamate successive), ma **nessun linkage** (il loro nome è accessibile solo dentro quella funzione).

2.3 Durata di Memorizzazione (Storage Duration §6.2.4)

1. **Statica:** All'avvio del programma nel Segmento Dati/BSS; persiste fino alla terminazione (globali e locali `static`).
2. **Automatica:** Creata sullo **Stack** all'ingresso del blocco e distrutta all'uscita (locali ordinarie e parametri).

3. **Dinamica:** Allocata esplicitamente sull'**Heap** (`malloc`, `free`).

2.4 Tabella Riepilogativa di Linkage per l'Esame

```
int a; // Definizione tentativa | Linkage Esterno
extern int b; // Dichiarazione | Linkage Esterno
static int c = 1; // Definizione | Linkage Interno
int cfun(int a, int b); // Dichiarazione | Linkage Esterno

static void my_func(int d) { // Definizione | Linkage Interno
    int e = 0; // Definizione | Nessun Linkage (automatica)
    static int f = 1; // Definizione | Nessun Linkage (durata
statica)
    extern int c; // Dichiarazione | Linkage Interno (eredita da
riga 3)
}
```

3. Precedenze, Sequence Points e Cortocircuito kaboom

3.1 Basi Numeriche e Letterali

- **Ottale (base 8):** Prefisso `0` (es. `07` = 7, `021` = 17, `073` = 59).
 - **Esadecimale (base 16):** Prefisso `0x` (es. `0xa` = 10, `0x14` = 20, `0x1E` = 30, `0xae` = 174).
-

3.2 Precedenze degli Operatori (dall'alta alla bassa)

1. `()`, `[]`, `→`, `.`, `a++`, `a--` (da sinistra a destra)
2. `++a`, `--a`, `!`, `~`, `+a`, `-a`, `*p`, `&a`, `(type)`, `sizeof` (da destra a sinistra)
3. `*`, `/`, `%`
4. `+`, `-`
5. `<<`, `>>`
6. `<`, `≤`, `>`, `≥`
7. `=`, `≠`
8. `&` (AND bitwise) `→` `^` (XOR) `→` `|` (OR bitwise)
9. `&&` (AND logico, cortocircuito)
10. `||` (OR logico, cortocircuito)
11. `? :` (operatore ternario, da destra a sinistra)

12. `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=` (da destra a sinistra)
13. `,` (operatore virgola, restituisce il valore destro)
-

3.3 Sequence Points, Side Effects e Undefined Behavior

- **Side Effect (Effetto Collaterale):** Modifica persistente dello stato di memoria (`a = 5`, `a++`, `--a`, `*p = 10`).
- **Sequence Point:** Punto di sincronizzazione in cui tutti i side effect precedenti sono stati applicati e nessuno dei successivi è ancora iniziato:
 - Fine istruzione (`;`).
 - Operatore virgola (`,`).
 - Operatori logici di cortocircuito (`&&`, `||`).
 - Operatore ternario (`? :`).
 - Punto prima dell'ingresso in una funzione chiamata.

Regola dell'Undefined Behavior (§6.5.2): Tra due sequence point consecutivi, una variabile scalare non può essere modificata più di una volta.

- `a = a++;` o `a++ + a++;` $\Rightarrow$ **Undefined Behavior / Warning del compilatore.**
 - `a++, a++;` o `a++ && a++;` $\Rightarrow$ **Valido** (la virgola e `&&` introducono un sequence point).
-

3.4 Cortocircuito Logico

- `A && B`: Se `A == 0` (falso), `B` **non viene valutato** (nessun effetto collaterale in `B`).
 - `A || B`: Se `A != 0` (vero), `B` **non viene valutato**.
-

4. Mappa di Memoria, Puntatori Multi-Tipo e Bitwise

4.1 Modello di Memoria e Little-Endian

- **Dimensioni standard:** `char` = 1 byte, `short` = 2 byte, `int` = 4 byte, `long long` = 8 byte.
- **Little-Endian:** Il **byte meno significativo (LSB)** viene memorizzato all'indirizzo più basso (all'indice/offset inferiore).

- Es: `int x = 0x00020004` → Byte 0: `0x04` (LSB), Byte 1: `0x00`, Byte 2: `0x02`, Byte 3: `0x00` (MSB).
 - **Complemento a due:**
 - `INT_MIN = -2147483648` (`0x80000000`), `INT_MAX = 2147483647` (`0x7FFFFFFF`).
 - `LLONG_MIN = 0x8000000000000000`, `LLONG_MAX = 0x7FFFFFFFFFFFFFFF`.
 - `(LLONG_MAX + 1)` a 64 bit fa overflow producendo `LLONG_MIN`.
-

4.2 Decodifica delle Espressioni nei Quesiti di Memoria

1. Accesso al Valore (Dereferenziazione `*` o `[...]` senza `&`):

- `q[k]` → legge 1 byte come `char` all'offset $+k$.
- `p[k]` → legge 2 byte come `short` a partire dall'offset $+2k$ (LSB a $2k$, MSB a $2k+1$).

$$\text{Valore} = \text{LSB} + 256 \times \text{MSB}$$

- `*((short*)&q[k])` → legge 2 byte all'offset $+k$ (LSB a k , MSB a $k+1$).

2. Indirizzi fisici (`&` o puntatore senza dereferenziazione):

- `&q[k]` → indirizzo del Byte k .
- `&p[k]` → indirizzo del Byte $2k$.
- `&a[k]` → indirizzo del Byte $k \times \text{sizeof}(a[0])$.

3. Sottrazione tra Puntatori vs Sottrazione con Cast `(int)`:

- `ptr1 - ptr2` (senza cast): Distanza in **numero di elementi**.

$$\&a[3] - a = 3 \quad (\text{elementi int}), \quad \&p[9] - \&p[2] = 7 \quad (\text{elementi short})$$

- `(int)ptr1 - (int)ptr2` (con cast esplicito): Distanza fisica esatta in **singoli BYTE**.
 - `(int)(a + 3) - (int)&q[6]` (con `int a[]`): $3 \times 4 - 6 = 12 - 6 = 6$ byte.
 - `(int)(p + 11) - (int)(a + 2)` (con `long long a[]` e `short *p`): $11 \times 2 - 2 \times 8 = 22 - 16 = 6$ byte.
-

4.3 Operazioni Bitwise

- `~x`: NOT bit a bit ($\sim 0x00 = 0xFF$).
- `a & b`: AND (1 solo se entrambi sono 1).
- `a | b`: OR (1 se almeno uno è 1).
- `a ^ b`: XOR (1 se discordi).
- `x >> k`: Shift a destra di k bit ($\lfloor x/2^k \rfloor$).
- `x << k`: Shift a sinistra di k bit ($x \times 2^k$).

5. Compilazione con GCC, Progetti Multi-File e Memoria

5.1 Pipeline GCC e Flag

`.c` $\xrightarrow{\text{Preprocessore [-E]}}$ `.i` $\xrightarrow{\text{Compilatore [-S]}}$ `.s` $\xrightarrow{\text{Assembler [-c]}}$ `.o` $\xrightarrow{\text{Linker [-o]}}$ Eseguibile

- `gcc -c file.c`: Produce solo il file oggetto `file.o`. **Il linker non viene eseguito → MAI errori di linker.**
- `gcc -o main main.c`: Esegue tutte le fasi. Dà **errore di linker** se mancano definizioni di funzioni chiamate (es. `undefined reference to mostra`).
- `gcc -o out out.c`: Dà **errore di linker** se `out.c` non contiene `main` (`undefined reference to main`).
- `gcc main.c out.c -o prog`: Compila e linka entrambi i moduli. Nessun errore se i file si completano a vicenda e le variabili `static` evitano collisioni.

5.2 Tracciamento Esecuzione Multi-File (Terminazione vs Ciclo Infinito)

1. **Verificare la variabile di controllo del `while` in `main()`:**
 - Se `main()` decrementa la variabile globale (es. `k -= 2;`), il ciclo **termina** regolarmente.
 - Se `main()` passa la variabile solo per valore alla funzione esterna e non la decrementa mai nel corpo del ciclo, la variabile globale di `main.c` resta invariata → **ciclo infinito**.
2. **Verificare le variabili `static` nel secondo file (`out.c`):**
 - Una variabile `static int k` definita in `out.c` mantiene il proprio stato tra le chiamate ma è **indipendente** dalla variabile `k` globale di `main.c`.

5.3 Le 4 Zone di Memoria di un Processo C



(malloc, calloc, realloc)



5.4 Gestione Dinamica della Memoria (<stdlib.h>)

- `malloc(size)` : Alloca `size` byte non inizializzati sull'Heap. Ritorna `NULL` se fallisce.
- `calloc(n, size)` : Alloca $n \times \text{size}$ byte e **azzerà tutti i byte a 0**.
- `realloc(ptr, new_size)` : Ridimensiona il blocco a `new_size` byte. Può spostare il blocco in una nuova area di memoria (restituendo un indirizzo diverso) o ritornare `NULL` in caso di errore.
- `free(ptr)` : Dealloca la memoria sull'Heap. Invocare `free()` su variabili automatiche di stack (es. `int a; free(&a);`) provoca **Undefined Behavior / Crash**.

5.5 Lvalue vs Rvalue

- **lvalue (Locator Value)** : Espressione che identifica una locazione di memoria modificabile (`a`, `*p`, `a[i]`, `*&a`).
- **rvalue (Read Value)** : Valore temporaneo di sola lettura privo di allocazione riassegnabile (`5`, `a + 2`, `&a`, `x && y`).

6. Liste Concatenate

6.1 Struttura e Pattern Ricorrenti

```
struct Node {  
    int info;  
    struct Node *pNext;  
};
```

1. Scansione con puntatore di supporto:

```
struct Node *curr = pFirst;  
while (curr != NULL) {  
    // elabora curr->info
```

```
    curr = curr->pNext;
}
```

2. Cancellazione sicura del nodo target:

```
struct Node *target = prev->pNext;
prev->pNext = target->pNext;
free(target);
```

3. Inversione in-place (3 puntatori):

```
void inverti_lista(void) {
    struct Node *prev = NULL, *curr = pFirst, *next = NULL;
    while (curr != NULL) {
        next = curr->pNext;
        curr->pNext = prev;
        prev = curr;
        curr = next;
    }
    pFirst = prev;
}
```

4. Partizione Stabile in-place (tecnica Head & Tail a due code):

- Mantiene due coppie di puntatori (dispH, dispT) e (pariH, pariT).
- Inserisce ogni elemento in coda alla rispettiva sottolista per preservare l'ordine relativo originale.
- Concatena: dispT->pNext = pariH; e aggiorna pFirst = (dispH != NULL) ? dispH : pariH; .

7. Matrici 2D, Variable-Length Arrays e Geometria bastarda

7.1 Layout Row-Major e VLA (C99)

- Le matrici in C sono memorizzate per righe contigue in memoria:

$$\text{Offset}(i, j) = (i \times \text{cols} + j) \times \text{sizeof}(\text{tipo})$$

- **Sintassi VLA:** Nei prototipi e parametri di funzione le variabili di dimensione devono **precedere** la matrice:

```
void elabora(int r, int c, int mat[r][c]); // Corretto
```

7.2 Formule di Mappatura Indici

Trasformazione	Dimensione Input → Output	Mappatura Indici
Diagonale Principale	$n \times n \rightarrow n$	<code>mat[i][i]</code>
Diagonale Secondaria	$n \times n \rightarrow n$	<code>mat[i][n - 1 - i]</code>
Trasposta	$m \times n \rightarrow n \times m$	<code>B[j][i] = A[i][j]</code>
Rotazione Antioraria 90°	$m \times n \rightarrow n \times m$	<code>B[n - 1 - j][i] = A[i][j]</code>
Rotazione Oraria 90°	$m \times n \rightarrow n \times m$	<code>B[j][m - 1 - i] = A[i][j]</code>
Rotazione 180°	$m \times n \rightarrow m \times n$	<code>B[m - 1 - i][n - 1 - j] = A[i][j]</code>